

CSE46901: Introduction to Robotics

Physical Milestone 3

Professor Jaeyeon Lee

TA: Minhyuk Park

Introduction

Welcome to CSE46901. We have prepared a few milestones before the final project, in which you will use a physical robot to execute a mission given by us. The following milestones will give you an idea how to interact with [ROBOTIS' Turtlebot3 waffle Pi](#) with a [Manipulator Arm](#) using the Robot Operating System.

The third milestone is about ROS programming. In this milestone, you will write a ROS program to control the OpenManipulator on the Turtlebot by keyboard. **The program should be similar to turtlebot3_teleop_key in milestone 1, but it will accept a few more keystrokes to control the robot arm.** Your program should provide **some basic functions such as setting the arm to some predefined poses and opening/closing the gripper.** In addition, your program **should read the current configuration of the arm and the gripper and display the information on the screen.** Of course, please feel free to add more functionalities to your program. You can write your program in either C++ or Python. Please **read the source code in these ROS packages you downloaded in the previous milestones in catkin_ws** to learn about how to use ROS to control the arm: turtlebot3_manipulation/ turtlebot3_manipulation_gui and turtlebot3/turtlebot3_teleop. Please also refer to the ROS programming books and the official ROS tutorial.

Submission requirement

Hardware video demonstration submission is required for milestone3. (i.e. **You may not demonstrate your code on simulation for a pass on this milestone.**) Nevertheless, you might be able to use a simulated milestone 2 environment to speed up development for the physical milestone 3 s. For detailed requirements, please refer to “**demo requirement**” at the end of this document.

Once your group is done with the **video demonstration**, please submit it on the **blackboard along with all of your source code**. Each group will submit one video and a zip file with your group name (i.e. group_name.zip) containing your source code. Rules for robot room usage will apply. **Every member will be asked to be present during receiving/handing over the turtlebot in the robot room** as our soft-policy to discourage free-riding.

Once we verify that you have submitted the milestone requirement, you will be awarded Pass/Fail. **You can resubmit videos to challenge your Fail status until the final project time period** with no penalty for resubmission. (i.e. no difference between teams who get it right the first try vs teams who retries). However, **do note that there is a logistical limit of each team being able to physically interact with the turtlebot.**

Example interface of your program

Control TurtleBot3 + OpenManipulatorX

8 : increase linear velocity
2 : decrease linear velocity
4 : increase angular velocity
6 : decrease angular velocity
5 : base stop

g : gripper open
f : gripper close

0 : Extend arm forward
1 : home pose

q to quit

Present Linear Velocity: 0.000, Angular Velocity: 0.000
Present Joint Angle J1: 0.000 J2: 0.000 J3: 0.000 J4: 0.000
Present Kinematics Position X: 0.000 Y: 0.000 Z: 0.000

Helper terminal code for c++ programmers (posix terminal)

```
#include <cstdio>
#include <iostream>
#include <limits>
#include <string>

#include <unistd.h>
#include <termios.h>
#include <poll.h>
#include <signal.h>

#include "../include/unist_tb3_kb/posix_terminal.hpp"

// This code is based on
// https://cboard.cprogramming.com/linux-programming/158476-termios-examples.html
// and
// http://www.cplusplus.com/forum/general/5304/#msg23940

// Usage:
//
// ....
// terminal_init();
// ....
// set_terminal_raw_mode();
// ....
// bool isContinue = true;
// while(isContinue) {
//     ....
//     while(true) {
//         if (wait_for_key_pressed(100)) break;
//         ....
//     }
//     ....
//     char c = getKeyStroke();
//     switch(c) {
//         ....
//         case 'q':
//             isContinue = false;
//             break;
//         ....
//     }
// }
// ....
// set_terminal_original_mode();
```

```

/* Restore terminal to original settings */
void terminal_done() {
    if (terminal_descriptor != -1)
        tcsetattr(terminal_descriptor, TCSANOW, &terminal_original);
}

/* "Default" signal handler: restore terminal, then exit. */
void terminal_signal(int signum) {
    cout << "Terminal_signal(" << signum << ")" << endl;
    if (terminal_descriptor != -1)
        tcsetattr(terminal_descriptor, TCSANOW, &terminal_original);
    /* exit() is not async-signal safe, but _exit() is.
     * Use the common idiom of 128 + signal number for signal exits.
     * Alternative approach is to reset the signal to default handler,
     * and immediately raise() it. */
    _exit(128 + signum);
}

/*
 * Initialize terminal for non-canonical, non-echo mode,
 * that should be compatible with standard C I/O.
 * Returns 0 if success, nonzero errno otherwise.
 */
int terminal_init() {

    /* Already initialized? */
    if (terminal_descriptor != -1)
        return errno = 0;

    /* Which standard stream is connected to our TTY? */
    if (isatty(STDERR_FILENO))
        terminal_descriptor = STDERR_FILENO;
    else
        if (isatty(STDIN_FILENO))
            terminal_descriptor = STDIN_FILENO;
        else
            if (isatty(STDOUT_FILENO))
                terminal_descriptor = STDOUT_FILENO;
            else
                return errno = ENOTTY;

    /* Obtain terminal settings. */
    if (tcgetattr(terminal_descriptor, &terminal_original) ||

```

```

    tcgetattr(descriptor, &terminal_settings))
{
    return errno = ENOTSUP;
}

// Disable buffering for terminal streams.
if (isatty(STDIN_FILENO))
    setvbuf(stdin, NULL, _IONBF, 0);
if (isatty(STDOUT_FILENO))
    setvbuf(stdout, NULL, _IONBF, 0);
if (isatty(STDERR_FILENO))
    setvbuf(stderr, NULL, _IONBF, 0);

/* At exit() or return from main(),
 * restore the original settings. */
if (atexit(terminal_done))
    return errno = ENOTSUP;

/* Set new "default" handlers for typical signals,
 * so that if this process is killed by a signal,
 * the terminal settings will still be restored first. */
struct sigaction act;
sigemptyset(&act.sa_mask);
act.sa_handler = terminal_signal;
act.sa_flags = 0;
if (sigaction(SIGHUP, &act, NULL) ||
    sigaction(SIGINT, &act, NULL) ||
    sigaction(SIGQUIT, &act, NULL) ||
    sigaction(SIGTERM, &act, NULL) ||
#ifdef SIGXCPU
    sigaction(SIGXCPU, &act, NULL) ||
#endif
#ifdef SIGXFSZ
    sigaction(SIGXFSZ, &act, NULL) ||
#endif
#ifdef SIGIO
    sigaction(SIGIO, &act, NULL) ||
#endif
    sigaction(SIGPIPE, &act, NULL) ||
    sigaction(SIGALRM, &act, NULL))
{
    return errno = ENOTSUP;
}

/* Let BREAK cause a SIGINT in input. */
terminal_settings.c_iflag &= ~IGNBRK;
terminal_settings.c_iflag |= BRKINT;

```

```

/* Ignore framing and parity errors in input. */
terminal_settings.c_iflag |= IGNPAR;
terminal_settings.c_iflag &= ~PARMRK;

/* Do not strip eighth bit on input. */
terminal_settings.c_iflag &= ~ISTRIP;

/* Do not do newline translation on input. */
// terminal_settings.c_iflag &= ~(INLCR | IGNCR | ICRNL);

#ifdef IUCLC
/* Do not do uppercase-to-lowercase mapping on input. */
terminal_settings.c_iflag &= ~IUCLC;
#endif

/* Use 8-bit characters. This too may affect standard streams,
 * but any sane C library can deal with 8-bit characters. */
terminal_settings.c_cflag &= ~CSIZE;
terminal_settings.c_cflag |= CS8;

/* Enable receiver. */
terminal_settings.c_cflag |= CREAD;

/* Let INTR/QUIT/SUSP/DSUSP generate the corresponding signals. */
terminal_settings.c_lflag |= ISIG;

/* Enable noncanonical mode.
 * This is the most important bit, as it disables line buffering etc. */
terminal_settings.c_lflag &= ~ICANON;

/* Disable echoing input characters. */
terminal_settings.c_lflag &= ~(ECHO | ECHOE | ECHOK | ECHONL);

/* Disable implementation-defined input processing. */
terminal_settings.c_lflag &= ~IEXTEN;

/* To maintain best compatibility with normal behaviour of terminals,
 * we set TIME=0 and MAX=1 in noncanonical mode. This means that
 * read() will block until at least one byte is available. */
terminal_settings.c_cc[VTIME] = 0;
terminal_settings.c_cc[VMIN] = 1;

/* Done. */
return errno = 0;
}

void set_terminal_raw_mode() {

```

```

/* Set the new terminal settings.
 * Note that we don't actually check which ones were successfully
 * set and which not, because there isn't much we can do about it. */
if (terminal_descriptor != -1)
    tcsetattr(terminal_descriptor, TCSANOW, &terminal_settings);
}

```

```

void set_terminal_original_mode() {
    /* Restore original terminal settings. */
    if (terminal_descriptor != -1)
        tcsetattr(terminal_descriptor, TCSANOW, &terminal_original);
}

```

```

bool wait_for_key_pressed(unsigned timeout_ms /* = 0 */) {
    if (terminal_descriptor == -1) return false;

    struct pollfd pls[ 1 ];
    pls[ 0 ].fd    = terminal_descriptor;
    pls[ 0 ].events = POLLIN | POLLPRI;

    return poll( pls, 1, timeout_ms ) > 0;
}

```

```

char getKeyStroke() {
    while(true) {
        char c = cin.get();
        if (c != 27) {
            return c;
        } else if (wait_for_key_pressed(50)) {
            c = cin.get();
            if (c == '[') {
                c = cin.get();
                switch(c) {
                    case 'A':
                        return UP_ARROW_KEY;
                    case 'B':
                        return DOWN_ARROW_KEY;
                    case 'C':
                        return LEFT_ARROW_KEY;
                    case 'D':
                        return RIGHT_ARROW_KEY;
                    default:
                        return ESC_KEY;
                }
            }
        }
    }
}

```

```
    } else {  
        return ESC_KEY;  
    }  
} else {  
    return ESC_KEY;  
}  
}  
}
```

Demo requirement

You will have to submit a video showing that you have successfully controlled the manipulator with **your own original program**. Your video must first briefly explain how your code works , functionally and structurally, and must show that your code have following functions.

- (1) Must be able to be run on a local terminal window or ssh.
(i.e. Do not program your own gui or use programs such as rviz)
- (2) Move turtlebot's wheels like teleop_node
- (3) Move the open manipulator robotic arm to a few predefined positions. Suggested positions are "Home position" and "Extend forward".
- (4) Open and close the gripper at the end of the open manipulator robotic arm.
- (5) Constantly display the current configuration of the arm and the gripper and display the information on the screen. The contents do not have to be the same as the example interface. Please demonstrate that you are able to subscribe to relevant topics to get information about the turtlebot. See "**Example interface of your program**" above for reference on what information to display on the screen.

To do this, ideally your code should maintain some stationary text on the terminal window and just change relevant numbers , as if stationary text are part of a ui element instead of scrolling new text up every update. See "**Posix terminal**" code above for c++ programmers. However, we are not going to penalize you should you fail to achieve this style of display.

You will have to submit the source code of your program as well as a video clip showing how your program works. Please submit one video and a zip file with your group name (i.e. group_name.zip) containing your source code.

Topic explanation is not required for physical milestones.